



MEMORIA PROCESADOR DE MYJS

Acedo Blanco, Alvaro
Czepiel, Daniel
Grupo 96

20 de enero de 2026

Índice

1. Diseño del analizador léxico	2
1.1. Tokens	2
1.2. Gramática	2
1.3. Diagrama del autómata y acciones semánticas	3
1.4. Descripción de las Acciones Semánticas	3
1.5. Errores	4
2. Diseño del analizador sintáctico	5
2.1. Gramática	5
2.2. Tabla	5
2.3. Errores	7
3. Diseño del analizador semántico	8
3.1. Errores	11
4. Diseño de la tabla de símbolos	12
5. Diseño del gestor de errores	13
6. Anexo: ficheros de prueba	14
6.1. Ficheros correctos	14
6.1.1. Fichero 1	14
6.1.2. Fichero 2	14
6.1.3. Fichero 3	14
6.1.4. Fichero 4	15
6.1.5. Fichero 5	15
6.2. Ficheros con errores	15
6.2.1. Fichero 1	15
6.2.2. Fichero 2	15
6.2.3. Fichero 3	16
6.2.4. Fichero 4	16
6.2.5. Fichero 5	16

El axioma es: S y las reglas de producción son:

$$P = \{ S \rightarrow \text{EOF} \mid \text{del } S \mid dA \mid cD \mid _D \mid "E \mid (\mid) \mid + \mid -I \mid > G \mid \\ \mid ! \mid \&H \mid = \mid ; \mid \{ \mid \} \mid , \mid / J$$

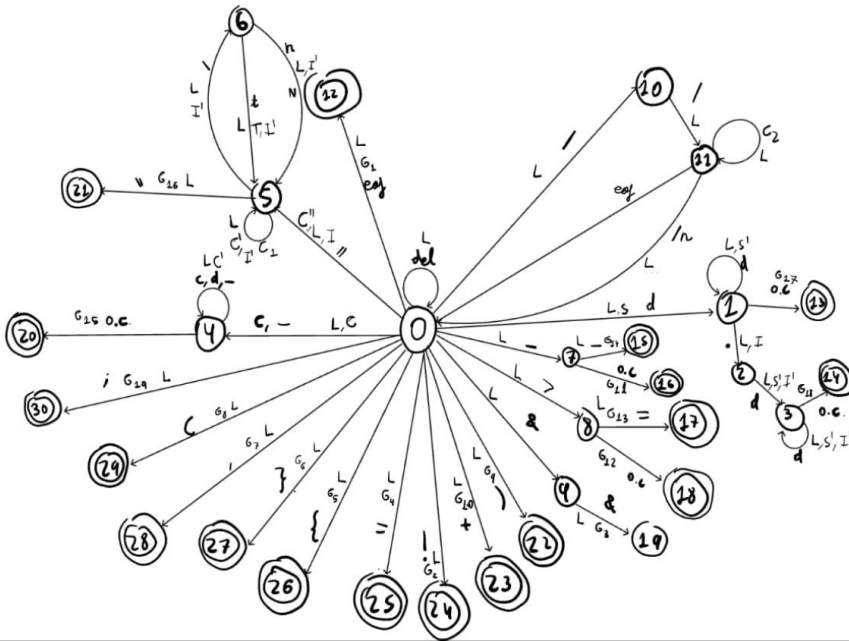
$$\begin{aligned} A &\rightarrow dA \mid .B \mid \lambda \\ B &\rightarrow dC \\ C &\rightarrow dC \mid \lambda \\ D &\rightarrow cD \mid_D dD \mid \lambda \\ E &\rightarrow c_1E \mid \backslash F \mid " \\ F &\rightarrow nE \mid tE \\ G &\rightarrow = \mid \lambda \\ H &\rightarrow \& \\ I &\rightarrow - \mid \lambda \\ J &\rightarrow / K \\ K &\rightarrow c_2K \mid CRS EOF \end{aligned}$$

Siendo:

$$\begin{aligned} \text{del} &= \{\text{espacio, TAB, CR}\} \\ c_1 &= T \setminus \{\backslash, \text{EOF, TAB, "}\} \\ c_2 &= T \setminus \{\text{EOF, TAB}\} \end{aligned}$$

1.3. Diagrama del autómata y acciones semánticas

Hicimos el siguiente autómata de la gramática regular:



Las acciones semánticas son las siguientes:

1.4. Descripción de las Acciones Semánticas

A continuación se detallan todas las acciones semánticas ejecutadas por el autómata durante el proceso de análisis léxico:

G1: Generar Token $\langle \text{eof, -} \rangle$

G2: Generar Token $\langle \text{not, -} \rangle$

G3: Generar Token $\langle \text{and, -} \rangle$

G4: Generar Token $\langle \text{asignación, -} \rangle$

G5: Generar Token $\langle \text{llaveApertura}, - \rangle$
G6: Generar Token $\langle \text{llaveCierre}, - \rangle$
G7: Generar Token $\langle \text{coma}, - \rangle$
G8: Generar Token $\langle \text{parentesisApertura}, - \rangle$
G9: Generar Token $\langle \text{parentesisCierre}, - \rangle$
G10: Generar Token $\langle \text{suma}, - \rangle$
G11: Generar Token $\langle \text{resta}, - \rangle$
G12: Generar Token $\langle \text{mayor}, - \rangle$
G13: Generar Token $\langle \text{mayorIgual}, - \rangle$
G14: Generar Token $\langle \text{autodecremento}, - \rangle$
G15: **if** $p \neq \text{null}$ **then**
 Generar Token $\langle \text{PR_p}, - \rangle$
 else
 $p = \text{BuscarTS}(\text{lexema})$
 if $p = \text{null}$ **then**
 $p = \text{InsertarTS}(\text{lexema})$
 Generar Token $\langle \text{identificador}, p \rangle$
G16: **if** $\text{cont} < 65$ **then**
 Generar Token $\langle \text{cadena}, \text{lexema} \rangle$
 else
 Error(28)
G17: **if** $2^{15} > \text{num}$ **then**
 Generar Token $\langle \text{entero}, \text{num} \rangle$
 else
 Error(21)
G18: $\text{num} = \text{num} \times 10^{-\text{cont}}$
 if $\text{num} < \text{NUM_MAX}$ **then**
 Generar Token $\langle \text{real}, \text{num} \rangle$
 else
 Error(24)
G19: Generar Token $\langle \text{puntoComa}, - \rangle$
L: $\text{car} = \text{leer}()$
C: $\text{lexema} = \text{caracter}$
C': $\text{lexema} = \text{lexema} + \text{caracter}$
C'': $\text{lexema} = ;$ $\text{cont} = 0$
N: $\text{lexema} = \text{lexema} + '\backslash n'$

1.5. Errores

Para los errores hemos tenido en cuenta:

- Caracter desconocido
- El uso incorrecto de comentarios. Del tipo //
- El uso incorrecto del operador and
- La terminación de las cadenas
- Secuencias de escape correctas
- Números reales bien formados
- Esceso de bytes para un tipo de dato

Al encontrarse el analizador léxico con un error este lo que hace es llamar a una función que se encarga de meter en pila el error con su código y mensaje de error.

2. Diseño del analizador sintáctico

2.1. Gramática

Para la gramática hemos utilizado una gramática descendente tabular. Esta tiene la siguiente forma:

El conjunto de de símbolos no terminales es:

NoTerminales = $\{A, B, C, D, E, E1, F, G, H, I, K, L, O, P, Q, R, R1, S, T, U, U1, V, W, X, Z\}$

El axioma es:

Axioma = P

Producciones $P = \{$

$E \rightarrow RE'$	(R1)	$S \rightarrow \text{id } G;$	
$E' \rightarrow \&\&RE'$	(R2)	$S \rightarrow \text{write } E;$	
$E' \rightarrow \lambda$	(R3)	$S \rightarrow \text{read id};$	
$R \rightarrow UR'$		$S \rightarrow \text{return } X;$	
$R' \rightarrow > UR'$	(R5)	$G \rightarrow = E$	(R30)
$R' \rightarrow > = UR'$		$G \rightarrow (L)$	
$R' \rightarrow \lambda$		$L \rightarrow EQ$	
$U \rightarrow VU'$		$L \rightarrow \lambda$	
$U' \rightarrow +VU'$		$Q \rightarrow, EQ$	
$U' \rightarrow -VU'$	(R10)	$Q \rightarrow \lambda$	(R35)
$U' \rightarrow \lambda$		$X \rightarrow E$	
$V \rightarrow +W$		$X \rightarrow \lambda$	
$V \rightarrow -W$		$B \rightarrow \text{if } (E)Z$	
$V \rightarrow !W$		$Z \rightarrow S$	
$V \rightarrow - - W$	(R15)	$Z \rightarrow \{C\}I$	(R40)
$V \rightarrow W$		$I \rightarrow \text{else } D$	
$W \rightarrow \text{id } O$		$I \rightarrow \lambda$	
$W \rightarrow (E)$		$D \rightarrow \text{if } (E)\{C\}I$	
$W \rightarrow \text{entero}$		$D \rightarrow \{C\}$	
$W \rightarrow \text{real}$	(R20)	$B \rightarrow \text{let } T \text{ id};$	(R45)
$W \rightarrow \text{cadena}$		$B \rightarrow S$	
$W \rightarrow \text{true}$		$T \rightarrow \text{int}$	
$W \rightarrow \text{false}$		$T \rightarrow \text{float}$	
$O \rightarrow (L)$		$T \rightarrow \text{boolean}$	
$O \rightarrow \lambda$	(R25)	$T \rightarrow \text{string}$	(R50)
$F \rightarrow \text{function } H \text{ id } (A)\{C\}$	(R51)		
$H \rightarrow T$			
$H \rightarrow \text{void}$			
$A \rightarrow T \text{ id } K$			
$A \rightarrow \text{void}$	(R55)		
$K \rightarrow, T \text{ id } K$			
$K \rightarrow \lambda$			
$C \rightarrow BC$			
$C \rightarrow \lambda$			
$P \rightarrow BP$	(R60)		
$P \rightarrow FP$			
$P \rightarrow \lambda$	(R62)		

2.2. Tabla

Se adjunta en la siguiente página la tabla del analizador sintáctico por motivos de su tamaño, está en su propia página y dada la vuelta para poder caber mejor.

Se tuvo que calcular el follow y next de las reglas para poder hacerla, y la tabla demuestra que la gramática es LL(1), ya que, de no serlo deberían de coincidir en la misma casilla dos o más reglas.

Cuadro 1: Tabla de Análisis Sintáctico LL(1) - Transcripción Literal

[illegible]

2.3. Errores

Para los errores hemos tenido en cuenta:

- Errores al leer terminales no esperados
- Empezar mal una sentencia
- No poner los atributos de una función
- Declarar de forma incorrecta una función
- No terminar la asignación de una variable
- Expresiones con operandos incorrectas
- El cierre correcto del archivo con un final de bloque o de sentencia

3. Diseño del analizador semántico

Hemos utilizado el siguiente EdT:

#	Producción	Acciones Semánticas
1	$E \rightarrow R E_1$	$E.tipo = \text{if}(E_1.tipo == \text{void} \vee R.tipo == E_1.tipo)$ then $R.tipo$ else tipo_error {AUX -= 2}
2	$E_1 \rightarrow \&\& R E_{1_1}$	$E_1.tipo = \text{if}(R.tipo == \text{logico} \wedge E_{1_1}.tipo \in \{\text{logico}, \text{void}\})$ then $R.tipo$ else tipo_error {AUX -= 3}
3	$E_1 \rightarrow \lambda$	$E_1.tipo = \text{void}$ {AUX -= 0}
4	$R \rightarrow U R_1$	$R.tipo = \text{if}(R_1.tipo == \text{void})$ then $U.tipo$ else if($U.tipo == R_1.tipo$) then logico else tipo_error {AUX -= 2}
5	$R_{1_1} \rightarrow > U R_{1_2}$	$R_{1_1}.tipo = \text{if}(U.tipo \in \{\text{real}, \text{entero}\} \wedge R_{1_2} == \text{void})$ then $U.tipo$ else tipo_error {AUX -= 3}
6	$R_{1_1} \rightarrow \geq U R_{1_2}$	$R_{1_1}.tipo = \text{if}(U.tipo \in \{\text{real}, \text{entero}\} \wedge R_{1_2} == \text{void})$ then $U.tipo$ else tipo_error {AUX -= 3}
7	$R_1 \rightarrow \lambda$	$R_1.tipo = \text{void}$ {AUX -= 0}
8	$U \rightarrow V U_1$	$U.tipo = \text{if}(U_1.tipo == \text{void} \vee V.tipo == U_1.tipo)$ then $V.tipo$ else tipo_error {AUX -= 2}
9	$U_1 \rightarrow + V U_{1_1}$	$U_1.tipo = \text{if}((V.tipo \in \{\text{entero}, \text{real}\} \wedge U_{1_1}.tipo == \text{void}) \vee V.tipo == U_{1_1}.tipo)$ then $V.tipo$ else tipo_error {AUX -= 3}
10	$U_1 \rightarrow - V U_{1_1}$	$U_1.tipo = \text{if}((V.tipo \in \{\text{entero}, \text{real}\} \wedge U_{1_1}.tipo == \text{void}) \vee V.tipo == U_{1_1}.tipo)$ then $V.tipo$ else tipo_error {AUX -= 3}
11	$U_1 \rightarrow \lambda$	$U_1.tipo = \text{void}$ {AUX -= 0}
12	$V \rightarrow + W$	$V.tipo = \text{if}(W.tipo \in \{\text{entero}, \text{real}\})$ then $W.tipo$ else tipo_error {AUX -= 2}
13	$V \rightarrow - W$	$V.tipo = \text{if}(W.tipo \in \{\text{entero}, \text{real}\})$ then $W.tipo$ else tipo_error {AUX -= 2}
Continúa en la siguiente página...		

#	Producción	Acciones Semánticas
14	$V \rightarrow ! W$	$V.tipo = \text{if}(W.tipo == \text{lógico}) \text{ then } tipo_logico$ $\text{else } tipo_error$ $\{AUX -= 2\}$
15	$V \rightarrow - - id$	$V.tipo = \text{if}(\text{buscarTipo}(id.pos) \in \{\text{real}, \text{entero}\})$ $\text{then } buscarTipo(id.pos)$ $\text{else } tipo_error$ $\{AUX -= 2\}$
16	$V \rightarrow W$	$V.tipo = W.tipo$ $\{AUX -= 1\}$
17	$W \rightarrow id O$	$W.tipo = \text{if}(O.tipo == \text{void}) \text{ then } buscarTipo(id.pos)$ $\text{else if}(\text{buscarParam}(id.pos) == O.parametros)$ $\text{then } buscarTipoRet(id.pos)$ $\text{else } tipo_error$ $\{AUX -= 2\}$
18	$W \rightarrow (E)$	$W.tipo = E.tipo$ $\{AUX -= 3\}$
19	$W \rightarrow \text{entero}$	$W.tipo = \text{entero}$ $\{AUX -= 1\}$
20	$W \rightarrow \text{real}$	$W.tipo = \text{real}$ $\{AUX -= 1\}$
21	$W \rightarrow \text{cadena}$	$W.tipo = \text{cadena}$ $\{AUX -= 1\}$
22	$W \rightarrow \text{true}$	$W.tipo = \text{lógico}$ $\{AUX -= 1\}$
23	$W \rightarrow \text{false}$	$W.tipo = \text{lógico}$ $\{AUX -= 1\}$
24	$O \rightarrow (L)$	$O.tipo = tipo_funcion$ $O.parametros = L.parametros$ $O.numParam = L.numeroParametros$ $\{AUX -= 3\}$
25	$O \rightarrow \lambda$	$O.tipo = \text{void}$ $\{AUX -= 0\}$
26	$S \rightarrow id G ;$	$S.tipo = \text{if}(\text{buscarTipo}(id.pos) == G.tipo == tipo_funcion)$ $\text{then if}(\text{buscarParam}(id.pos) \neq$ $G.parametros) \text{ tipo_error}$ $\text{else if}(\text{buscarTipo}(id.pos) \neq G.tipo) \text{ then } tipo_error$ $\{AUX -= 3\}$
27	$S \rightarrow \text{write } E ;$	$S.tipo = \text{if}(E.tipo \notin$ $\{\text{entero}, \text{real}, \text{cadena}\}) \text{ then } tipo_error$ $S.tipoRet = \text{void}$ $\{AUX -= 3\}$
28	$S \rightarrow \text{read } id ;$	$S.tipo = \text{if}(\text{buscarTipoTS}(id.pos) \notin \{\text{entero}, \text{real}, \text{cadena}\})$ $\text{then } tipo_error$ $S.tipoRet = \text{void}$ $\{AUX -= 3\}$
29	$S \rightarrow \text{return } X ;$	$S.tipo = \text{if}(X.tipo \neq S.tipoRet) \text{ then } tipo_error$ $\{AUX -= 3\}$
30	$G \rightarrow = E$	$G.tipo = E.tipo$ $\{AUX -= 2\}$
31	$G \rightarrow (L)$	$G.tipo = tipo_funcion$ $G.parametros = L.parametros, G.numParam =$ $L.numParametros$ $\{AUX -= 3\}$
Continúa en la siguiente página...		

#	Producción	Acciones Semánticas
32	$L \rightarrow E Q$	$L.param = E.tipo \times Q.param, L.numP = 1 + Q.numP$ {AUX -= 2}
33	$L \rightarrow \lambda$	$L.param = \text{void}, L.numP = 0$ {AUX -= 0}
34	$Q_1 \rightarrow, E Q_2$	$Q_1.param = E.tipo \times Q_2.param, Q_1.numP = 1 + Q_2.numP$ {AUX -= 3}
35	$Q \rightarrow \lambda$	$Q.param = \text{void}, Q.numP = 0$ {AUX -= 0}
38, 39	$B \rightarrow \text{if } (E) Z$	$B.tipo = \text{if}(E.tipo == \text{logico}) \text{ then } tipo_error$ $B.tipoRet = Z.tipoRet$ {AUX -= 5}
51, 52	$B \rightarrow \text{let } T \text{ id ;}$	zonaDecl = true añadirTipo(id.pos, T.tipo) añadirDespl(id.pos, despl) desplazamiento+ = T.tamaño zonaDecl = false {AUX -= 4}
58-60	$F \rightarrow \text{function } H \text{ id } (A) \{ C \}$	zonaDecl = true, añadeTipo(id.pos, tipo_func) añadeTipoRet(id.pos, H.tipo), C.tipoRet = H.tipo añadeEtiqu(id.pos, nuevaEtiqu()) TSF = crearTabla(), despl = 0, zonaDecl = false añadeParam(id.pos, A.param), liberarTabla(TSF) {AUX -= 9}
68, 69	$PP \rightarrow P$	TS = crearTabla(), desplazamiento = 0 liberarTabla(TS) {AUX -= 1}

3.1. Errores

Para los errores hemos tenido en cuenta:

- El uso de tipos correctos en expresiones como and o >
- La no conversión de tipo en operaciones
- La utilización de llamadas a funciones con el número y el tipo correctos de los parámetros
- La asignación de variables con datos del mismo tipo
- Utilización de write y read con variables de tipo entero, real o cadena
- El tipo devuelto en funciones
- Que las sentencias de comprobación del if sean de tipo lógico

4. Diseño de la tabla de símbolos

En la tabla de símbolos hemos utilizado la búsqueda lineal para recorrer las tablas.

Hemos diferenciado las posiciones de la tabla de símbolos global de las de la función denotando las posiciones de la tabla de función negativas y haciendo que la primera posición de cada tabla sea 1 y -1 respectivamente.

5. Diseño del gestor de errores

Para el gestor de errores hemos decidido hacer recuperación de errores, así el gestor recopila los errores del archivo entero y luego los vuelca una vez terminado de analizar el fichero entero.

Para el contéo del número de línea, creamos una función la cual el analizador léxico llama cada vez que lee el delimitador salto de línea.

6. Anexo: ficheros de prueba

El lenguaje que elegimos para hacer la práctica es C, el cual te obliga a hacer gestión de la memoria dinámica. Hemos tenido en la implementación final varios errores de memoria.

6.1. Ficheros correctos

6.1.1. Fichero 1

```
1  let int a ;
2  let float b ;
3  let boolean bbb ;
4  a = a - c;
5  write a ;
6  write b;
7  write ccc;
```

fichero1.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

6.1.2. Fichero 2

```
1  let int x; function float hola (int hola){}
2  hola(x);
3  function void hola2(float f, boolean b){
4      let int i;
5      hola(--i);
6      hola(-i);
7      return ;
8  }
9
10 let boolean y;
11 hola2(hola(x),y);
```

fichero2.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

6.1.3. Fichero 3

```
1  function string hello(void){
2      let string hola = "hola";
3      write hola;
4      if(true){
5          return hola;
6      }
7      else{
8          return hello();
9      }
10 }
```

fichero3.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

6.1.4. Fichero 4

```
1 let boolean a;  
2 let boolean b;  
3 a = 3 &&( 4 > (2 + 1) )&& !true;  
4 b=a;  
5 let float rrr;  
6 let float ccc;if (rrr >= ccc) b = false;
```

fichero4.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

6.1.5. Fichero 5

```
1 //f y g no declarados antes, luego son tipo entero  
2 write (f - g);  
3 if(false){  
4     hello();  
5 }  
6 else if(true){  
7     nada = 4;  
8 }  
9 else{}  
10 if (true) read nada;
```

fichero5.txt

TablaTokens:

TablaSimbolos:

Parse:

6.2. Ficheros con errores

6.2.1. Fichero 1

```
1 //Creo que falta poner el void  
2 function float a fun (){  
3 let string tab = 9;  
4 return c;  
5 }  
6 (tab);
```

fichero1.txt

Errores:

6.2.2. Fichero 2

```
1 function int h (int a, float b, string c){  
2     if(true){  
3         --c;  
4         return c;  
5     }  
6     else{  
7         return h(c, b, a);  
8     }  
9 }
```

fichero2.txt

Errores:

6.2.3. Fichero 3

```
1 function string hola(float a,b){  
2   pues__nada = hola(a,b);  
3   let float a;  
4   let int b;  
5   if(a < b){}  
6   else if(!(a-b)){}  
}
```

fichero3.txt

Errores:

6.2.4. Fichero 4

```
1 if(a){  
2   return a;  
3 }  
4 if(3>2>1) then pues__nada = true;  
5 let int b;  
6 let float c;  
7 a=b+c;
```

fichero4.txt

Errores:

6.2.5. Fichero 5

```
1 function int entero (int a, float b){  
2   let string c;  
3   return c;  
4   return a-b;  
5   return ;  
6 }
```

fichero5.txt

Errores: